コンピュータサイエンス第一 第五回課題解答解説

今回のポイント

- 探索問題: リストから目的の値を探してインデックスを返す (なければ `None`)
- 逐次探索: 先頭から順に 1 つずつ比べる
- 2 分探索: 整列済みリストを半分ずつに絞りながら探す
- 計算量の感覚: 要素数 `n` に対して、逐次探索は `n` ステップ、2 分探索は $\log_2 n$ ステップ

各問の解答解説

問 1

```
def find_slow(data, item):
    for i in range(len(data)):
        if data[i] == item:
            return i
    return None
```

これによって、`data` の中に `item` と等しい値があれば**その最初のインデックス**、なければ `None` が返ってきます。例えば `find_slow([5,4,3,1,2,3], 1)` は `3`、`find_slow([1,2,3,4,5], 10)` は `None` です。プログラムの流れとしては、`for i in range(len(data))` で、`i` に 0 から最後の添字までを順番に入れて繰り返します。各回で `data[i] == item`、つまり `i` 番目の要素が探している値と等しいかを調べ、等しければ `return i` でその場で終了します。最後まで等しい要素が見つからなければループを抜けて、`return None` が実行されます。

細かい解説

> `find_slow([1,2,2,2,2,3], 2)` は、先頭から順に調べて最初に見つかった時点で `return` するので、`1` が返ります (`2` は添字 1〜4 にありますが、一番手前の 1 だけを返します)。`return None` は `for` の外 (インデントが一段浅い位置) に書きます。ループの中に書くと、最初の要素が違うだけで `None` を返して終わってしまいます。空のリスト `[]` では `range(0)` が空なので、ループは 1 回も実行されず、そのまま `None` になります。`find_slow` は、探している値がリストにないとき、すべての要素を比べてからでないと「ない」と言えません。要素が 100 万個なら 100 万ステップ必要です (確認問題の答えのとおりです)。

注. `index()`、`find()`、`in` 演算子は使えません。ただし `for ... in ...:` の `in` は演算子ではなく `for` 文の一部なので、使って大丈夫です。

問 2

```python
def find_quick(ordered, item):
    low = 0
    high = len(ordered) - 1
    while low <= high:
        mid = (low + high) // 2
        if ordered[mid] == item:
            return mid
        elif ordered[mid] < item:
            low = mid + 1
        else:
            high = mid - 1
    return None
```

これによって、昇順に整列されたリスト `ordered` から `item` を高速に探し、見つかればそのインデックス、なければ **None** が返ってきます。例えば `find_quick([1,2,3,4,5], 3)` は **2**、`find_quick([1,2,3,4,5], 10)` は **None** です。考え方は「探す範囲を半分ずつに絞る」ことです。`low` と `high` で探す範囲の両端(添字)を持ち、最初はリスト全体(`0` から `len(ordered) - 1`)にします。`while low <= high` の間、範囲の真ん中の添字 `mid = (low + high) // 2` の要素と `item` を比べます。

- 等しければ、見つかったので `return mid` で終了します。
- `ordered[mid] < item` なら、探す値は `mid` より右側にあるので、`low = mid + 1` として左半分を捨てます。
- それ以外(`ordered[mid] > item`)なら、探す値は `mid` より左側にあるので、`high = mid - 1` として右半分を捨てます。

範囲が空(`low > high`)になっても見つからなければ、リストに存在しないのでループを抜けて `return None` です。

細かい解説

`find_quick([1,2,3,4,5], 5)` の流れを追ってみましょう。

```
low=0 high=4  mid=2  ordered[2]=3 < 5  → low=3
low=3 high=4  mid=3  ordered[3]=4 < 5  → low=4
low=4 high=4  mid=4  ordered[4]=5 == 5 → return 4
```

`mid` の計算の`//`は整数の割り算です。`/`だと小数になって添字に使えません。`low = mid +`

> `1`、`high = mid - 1` のように `mid` 自身を範囲から外すのがポイントです。`mid` のままにすると、範囲が縮まらずに無限ループになることがあります。ループの条件が `low < high` ではなく `low <= high` なのは、範囲に要素が 1 つだけ残った状態（`low == high`）でも、その 1 つを調べる必要があるからです。同じ値が複数あるとき（例えば `[1,2,2,2,2,3]` の `2`）、2 分探索は**どれか 1 つ**のインデックスを返します（この例では `2`）。問題文は「`item` と等しい値のインデックス」としか求めていないので、これで構いません。最初のインデックスが必要な場合は、`find_slow` か、見つけた後にさらに左側を調べる工夫が必要です。確認問題の答えのとおり、2 分探索は 1 ステップごとに範囲が半分になるので、100 万個のリストでも約 20 ステップ（$2^{20} \approx 100$ 万）で結論が出ます。逐次探索の 100 万ステップとは大違いです。

注. 2 分探索は、リストが**昇順に整列されている**ことが前提です。整列されていないリストに使うと、正しい位置にあっても見つからないことがあります。